

LAPORAN TUGAS 6 - STRUKTUR SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Mohammad Putra Fernanda (2410651049)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

Tugas 1: Analisis Deadlock

Jalankan program deadlock_simple.c dan jelaskan:

1. Mengapa program mengalami deadlock?
2. Kondisi deadlock apa saja yang terpenuhi?
3. Gambar diagram resource allocation graph-nya

Jawab:

1. Program mengalami deadlock karena:

-Thread 1 mengunci lock1 terlebih dahulu, lalu mencoba mengambil lock2.

-Thread 2 mengunci lock2 terlebih dahulu, lalu mencoba mengambil lock1.

-Setelah itu, kedua thread saling menunggu mutex yang dipegang thread lainnya, dan tidak ada yang melepaskan kunci, sehingga program macet selamanya.

Ini menciptakan circular wait:

Thread 1 → menunggu lock2 (yang dipegang Thread 2)

Thread 2 → menunggu lock1 (yang dipegang Thread 1)

2. Kondisi deadlock apa saja yang terpenuhi?

Menurut teori Coffman, deadlock terjadi jika keempat kondisi berikut terpenuhi secara bersamaan. Semua kondisi ini muncul pada program:

- (1) Mutual Exclusion (Eksklusi Saling) – TERPENUHI

Resource (mutex) hanya bisa dipakai satu thread pada suatu waktu.
Mutex tidak dapat di-share, hanya satu thread bisa memegang satu mutex.

- (2) Hold and Wait (Memegang dan Menunggu) – TERPENUHI

-Thread 1 memegang lock1 sambil menunggu lock2.

-Thread 2 memegang lock2 sambil menunggu lock1.

- (3) No Preemption (Tidak Bisa Diambil Paksa) – TERPENUHI

Resource (mutex) tidak bisa diambil secara paksa oleh thread lain.
Thread hanya dapat melepaskan mutex secara sukarela.

(4) Circular Wait (Menunggu Melingkar) – TERPENUHI

Urutan menunggu membentuk lingkaran:

Thread 1 → menunggu lock2

Thread 2 → menunggu lock1

Kesimpulan:

Semua 4 kondisi deadlock terpenuhi → DEADLOCK pasti terjadi.

3. T1 → Lock2 → T2 → Lock1 → T1

Tugas 2: Implementasi Pencegahan

Modifikasi program deadlock_simple.c menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

Jawab:

1.

```
putrafernanda@DESKTOP-TTMU011:~$ nano deadlock_simple.c
putrafernanda@DESKTOP-TTMU011:~$ gcc deadlock_simple.c -o deadlock_simple -lpthread
putrafernanda@DESKTOP-TTMU011:~$ ./deadlock_simple
Thread 1: Mengambil Lock 1...
Thread 2: Mengambil Lock 1...
Thread 1: Mengambil Lock 2...
Thread 1: Selesai bekerja
Thread 2: Mengambil Lock 2...
Thread 2: Selesai bekerja
putrafernanda@DESKTOP-TTMU011:~$ |
```

Hasil:

-Tidak terjadi deadlock.

-Program selalu selesai.

Source code:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
// Thread 1
```

```
void* thread1_func(void* arg) {
```

```
    printf("Thread 1: Mengambil Lock 1...\n");
```

```
    pthread_mutex_lock(&lock1);
```

```
    sleep(1);
```

```
    printf("Thread 1: Mengambil Lock 2...\n");
```

```
    pthread_mutex_lock(&lock2);
```

```
    printf("Thread 1: Selesai bekerja\n");
```

```
    pthread_mutex_unlock(&lock2);
```

```
    pthread_mutex_unlock(&lock1);
```

```
    return NULL;
```

```
}
```

```
// Thread 2 (URUTAN SAMA!)
```

```
void* thread2_func(void* arg) {
```

```
    printf("Thread 2: Mengambil Lock 1...\n");
```

```
    pthread_mutex_lock(&lock1);
```

```
    sleep(1);
```

```
    printf("Thread 2: Mengambil Lock 2...\n");
```

```
    pthread_mutex_lock(&lock2);
```

```
    printf("Thread 2: Selesai bekerja\n");
```

```
    pthread_mutex_unlock(&lock2);
```

```
    pthread_mutex_unlock(&lock1);
```

```
    return NULL;
```

```
}
```

```
int main() {
```

```
    pthread_t t1, t2;
```

```
    pthread_create(&t1, NULL, thread1_func, NULL);
```

```
    pthread_create(&t2, NULL, thread2_func, NULL);
```

```

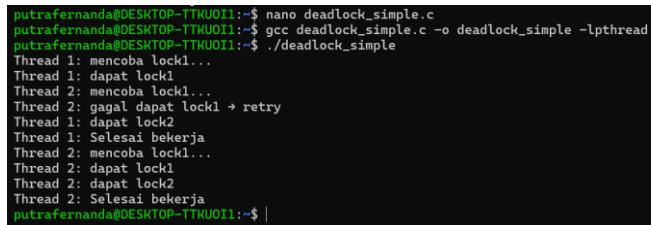
pthread_join(t1, NULL);

pthread_join(t2, NULL);

return 0;
}

```

2.



```

putrafernanda@DESKTOP-TTKU011:~$ nano deadlock_simple.c
putrafernanda@DESKTOP-TTKU011:~$ gcc deadlock_simple.c -o deadlock_simple -lpthread
putrafernanda@DESKTOP-TTKU011:~$ ./deadlock_simple
Thread 1: mencoba lock1...
Thread 1: dapat lock1
Thread 2: mencoba lock1...
Thread 2: gagal dapat lock1 -> retry
Thread 1: dapat lock2
Thread 1: Selesai bekerja
Thread 2: mencoba lock1...
Thread 2: dapat lock1
Thread 2: dapat lock2
Thread 2: Selesai bekerja
putrafernanda@DESKTOP-TTKU011:~$

```

Hasil:

- Tidak terjadi deadlock.
- Namun thread bisa retry berkali-kali sebelum berhasil.
- Ada kemungkinan starvation jika satu thread terus kalah rebutan lock.

source code:

```

#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread_func(void* arg) {

```

```
int id = *(int*)arg;

while (1) {

    printf("Thread %d: mencoba lock1...\n", id);

    if (pthread_mutex_trylock(&lock1) == 0) {

        printf("Thread %d: dapat lock1\n", id);

        usleep(100000); // 0.1 detik

        if (pthread_mutex_trylock(&lock2) == 0) {

            printf("Thread %d: dapat lock2\n", id);

            printf("Thread %d: Selesai bekerja\n", id);

            pthread_mutex_unlock(&lock2);

            pthread_mutex_unlock(&lock1);

            break;

        } else {

            printf("Thread %d: gagal dapat lock2 → melepas lock1\n", id);

            pthread_mutex_unlock(&lock1);

            usleep(100000);

        }

    } else {

        printf("Thread %d: gagal dapat lock1 → retry\n", id);
```

```

        usleep(100000);
    }
}

return NULL;
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    pthread_create(&t1, NULL, thread_func, &id1);
    pthread_create(&t2, NULL, thread_func, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}

```

3. Lock ordering mencegah deadlock dengan memastikan semua thread mengambil lock dalam urutan yang sama. Cara ini sangat sederhana, cepat, dan tidak membebani CPU, tetapi hanya efektif jika urutan resource bisa ditentukan dengan jelas sejak awal.

Sementara itu, try-lock dengan retry lebih fleksibel karena tidak memerlukan urutan lock. Jika gagal mengambil lock, thread melepas lock yang sudah didapat dan mencoba lagi sehingga deadlock tidak terjadi. Namun, metode ini bisa

membuat thread sering retry, membebani CPU, dan berpotensi menyebabkan starvation.

Secara umum, lock ordering lebih cocok untuk sistem kecil dan terstruktur, sedangkan try-lock lebih cocok untuk sistem besar yang kompleks dan sulit menentukan urutan lock.

Tugas 3: Banker's Algorithm

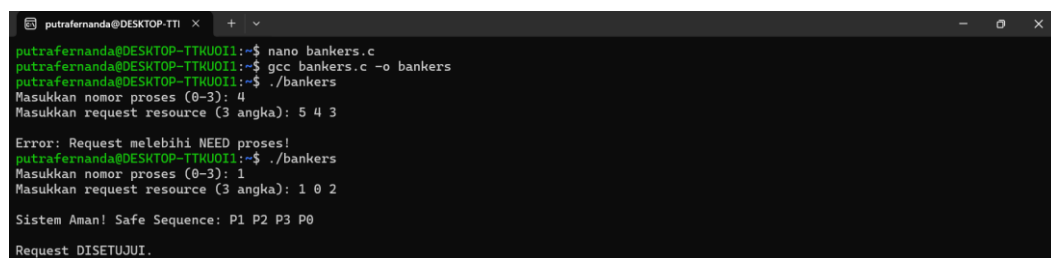
Buat program baru dengan data:

4 proses

3 jenis resource

Available: [5, 4, 3]

Implementasikan request resource dan cek keamanan system



```
putrafernanda@DESKTOP-TT1 X + v
putrafernanda@DESKTOP-TT1:~$ nano bankers.c
putrafernanda@DESKTOP-TT1:~$ gcc bankers.c -o bankers
putrafernanda@DESKTOP-TT1:~$ ./bankers
Masukkan nomor proses (0-3): 4
Masukkan request resource (3 angka): 5 4 3
Error: Request melebihi NEED proses!
putrafernanda@DESKTOP-TT1:~$ ./bankers
Masukkan nomor proses (0-3): 1
Masukkan request resource (3 angka): 1 0 2
Sistem Aman! Safe Sequence: P1 P2 P3 P0
Request DISETUJUI.
```

Source code:

```
#include <stdio.h>
```

```
#define P 4 // jumlah proses
```

```
#define R 3 // jumlah resource
```

```
int available[R] = {5, 4, 3};
```

```
int max[P][R] = {
```

```
{7, 5, 3},  
  
{3, 2, 2},  
  
{9, 0, 2},  
  
{2, 2, 2}  
};
```

```
int allocation[P][R] = {  
  
    {0, 1, 0},  
  
    {2, 0, 0},  
  
    {3, 0, 2},  
  
    {2, 1, 1}  
};
```

```
int need[P][R];
```

```
// Hitung matriks Need = Max - Allocation
```

```
void calculateNeed() {  
  
    for (int i = 0; i < P; i++)  
  
        for (int j = 0; j < R; j++)  
  
            need[i][j] = max[i][j] - allocation[i][j];  
  
}
```

```
// Fungsi cek apakah sistem dalam keadaan aman
```

```
int isSafe() {  
  
    int work[R];
```

```

int finish[P] = {0};

int safeSeq[P];

int count = 0;


for (int i = 0; i < R; i++)

    work[i] = available[i];


while (count < P) {

    int found = 0;


    for (int i = 0; i < P; i++) {

        if (!finish[i]) {

            int canRun = 1;

            for (int j = 0; j < R; j++) {

                if (need[i][j] > work[j]) {

                    canRun = 0;

                    break;

                }

            }

            if (canRun) {

                for (int j = 0; j < R; j++)

                    work[j] += allocation[i][j];

                safeSeq[count++] = i;

```

```

        finish[i] = 1;

        found = 1;

    }

}

}

if (!found)

    return 0; // unsafe

}

printf("\nSistem Aman! Safe Sequence: ");

for (int i = 0; i < P; i++)

    printf("P%d ", safeSeq[i]);

printf("\n");

return 1;

}

int main() {

    calculateNeed();

    int p;

    int request[R];

    printf("Masukkan nomor proses (0-3): ");

```

```
scanf("%d", &p);
```

```
printf("Masukkan request resource (3 angka): ");
```

```
for (int i = 0; i < R; i++)
```

```
    scanf("%d", &request[i]);
```

```
// Cek apakah request <= need
```

```
for (int i = 0; i < R; i++) {
```

```
    if (request[i] > need[p][i]) {
```

```
        printf("\nError: Request melebihi NEED proses!\n");
```

```
        return 0;
```

```
    }
```

```
}
```

```
// Cek apakah request <= available
```

```
for (int i = 0; i < R; i++) {
```

```
    if (request[i] > available[i]) {
```

```
        printf("\nResource tidak cukup (REQUEST > AVAILABLE)\n");
```

```
        return 0;
```

```
    }
```

```
}
```

```
// Simulasi alokasi sementara
```

```
for (int i = 0; i < R; i++) {
```

```
    available[i] -= request[i];
```

```

        allocation[p][i] += request[i];

        need[p][i] -= request[i];
    }

    // Cek keadaan aman atau tidak

    if (isSafe()) {
        printf("\nRequest DISETUJUI.\n");
    } else {
        printf("\nSistem tidak aman! Request DITOLAK.\n");

        // Undo perubahan

        for (int i = 0; i < R; i++) {
            available[i] += request[i];

            allocation[p][i] -= request[i];

            need[p][i] += request[i];
        }
    }

    return 0;
}

```

Tugas 4: Kasus Nyata (Real Case)

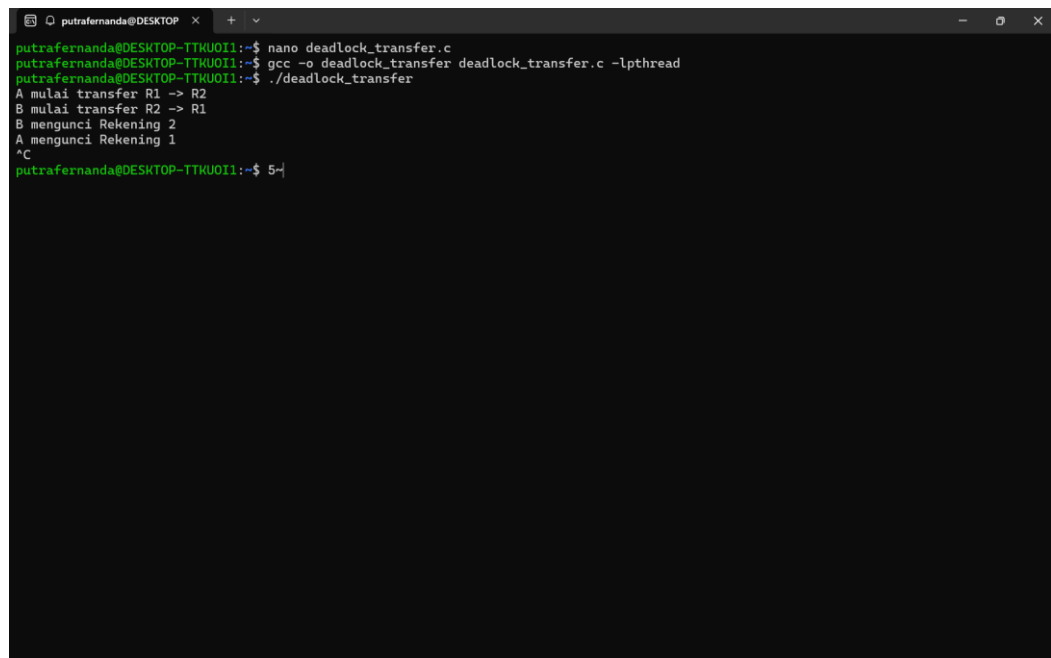
Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

Orang A: transfer dari Rekening 1 ke Rekening 2

Orang B: transfer dari Rekening 2 ke Rekening 1

Implementasikan deadlock prevention

(Simulasi deadlock pada dua rekening)



```
putrafernanda@DESKTOP-TTKUOI1:~$ nano deadlock_transfer.c
putrafernanda@DESKTOP-TTKUOI1:~$ gcc -o deadlock_transfer deadlock_transfer.c -lpthread
putrafernanda@DESKTOP-TTKUOI1:~$ ./deadlock_transfer
A mulai transfer R1 -> R2
B mulai transfer R2 -> R1
B mengunci Rekening 2
A mengunci Rekening 1
^C
putrafernanda@DESKTOP-TTKUOI1:~$
```

Source code:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
int rekening1 = 1000;
```

```
int rekening2 = 1000;
```

```
void* transfer_A(void* arg) {
```

```
printf("A mulai transfer R1 -> R2\n");

pthread_mutex_lock(&lock1);

printf("A mengunci Rekening 1\n");

sleep(1);

pthread_mutex_lock(&lock2); // DEADLOCK terjadi di sini

printf("A mengunci Rekening 2\n");

rekening1 -= 100;

rekening2 += 100;

pthread_mutex_unlock(&lock2);

pthread_mutex_unlock(&lock1);

return NULL;
}

void* transfer_B(void* arg) {

printf("B mulai transfer R2 -> R1\n");

pthread_mutex_lock(&lock2);

printf("B mengunci Rekening 2\n");

sleep(1);
```



```
pthread_mutex_lock(&lock1); // DEADLOCK menunggu A

printf("B mengunci Rekening 1\n");

rekening2 -= 100;

rekening1 += 100;

pthread_mutex_unlock(&lock1);

pthread_mutex_unlock(&lock2);

return NULL;
}

int main() {

pthread_t t1, t2;

pthread_create(&t1, NULL, transfer_A, NULL);

pthread_create(&t2, NULL, transfer_B, NULL);

pthread_join(t1, NULL);

pthread_join(t2, NULL);

printf("Saldo akhir: R1=%d, R2=%d\n", rekening1, rekening2);
}
```

Mencegah deadlock dengan Lock Ordering (mengunci berdasarkan ID terkecil terlebih dahulu)

```
putrafernanda@DESKTOP-TTKU011:~$ nano prevention_transfer.c
putrafernanda@DESKTOP-TTKU011:~$ gcc -o prevention_transfer prevention_transfer.c -lpthread
putrafernanda@DESKTOP-TTKU011:~$ ./prevention_transfer
A mulai transfer R1 -> R2
B mulai transfer R2 -> R1
Saldo akhir: R1=1000, R2=1000
putrafernanda@DESKTOP-TTKU011:~$ |
```

Source code:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
int rekening1 = 1000;
```

```
int rekening2 = 1000;
```

```
void lock_ordered(pthread_mutex_t* a, pthread_mutex_t* b) {
```

```
    if (a < b) {
```

```
        pthread_mutex_lock(a);
```

```
        pthread_mutex_lock(b);
```

```
    } else {
```

```
        pthread_mutex_lock(b);
```

```
        pthread_mutex_lock(a);
```

```
    }
```

```
}
```

```
void unlock_ordered(pthread_mutex_t* a, pthread_mutex_t* b) {
```

```
    if (a < b) {
```

```
        pthread_mutex_unlock(b);
```

```
        pthread_mutex_unlock(a);
```

```
    } else {
```

```
        pthread_mutex_unlock(a);
```

```
        pthread_mutex_unlock(b);
```

```
    }
```

```
}
```

```
void* transfer_A(void* arg) {
```

```
    printf("A mulai transfer R1 -> R2\n");
```

```
    lock_ordered(&lock1, &lock2);
```

```
    rekening1 -= 100;
```

```
    rekening2 += 100;
```

```
    unlock_ordered(&lock1, &lock2);
```

```
    return NULL;
```

```
}
```

```
void* transfer_B(void* arg) {

    printf("B mulai transfer R2 -> R1\n");

    lock_ordered(&lock1, &lock2);

    rekening2 -= 100;

    rekening1 += 100;

    unlock_ordered(&lock1, &lock2);

    return NULL;
}

int main() {

    pthread_t t1, t2;

    pthread_create(&t1, NULL, transfer_A, NULL);

    pthread_create(&t2, NULL, transfer_B, NULL);

    pthread_join(t1, NULL);

    pthread_join(t2, NULL);

    printf("Saldo akhir: R1=%d, R2=%d\n", rekening1, rekening2);

}
```